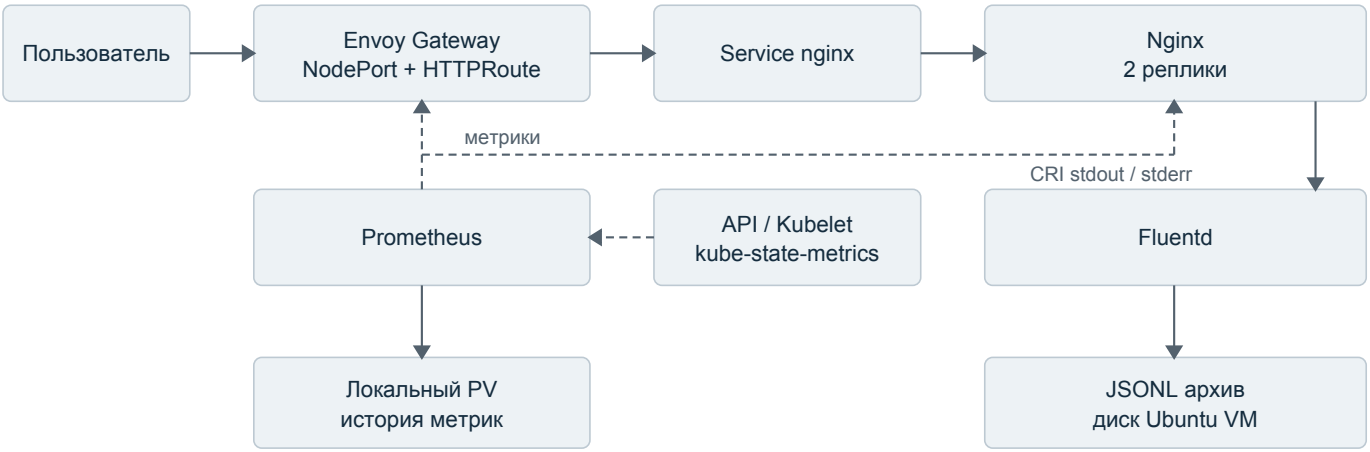
Паспорт DevOps решения

MTC ENGINEER HACK | Состояние на 4 октября 2026 года

Решение автоматизирует одноузловой стенд: веб-приложение в Kubernetes, публикацию через Gateway API, мониторинг Prometheus и сбор логов Fluentd. Для воспроизведения нужна отдельная Ubuntu Server 24.04; коммерческие сервисы не требуются.

Статус проверки. Локальные тесты и синтаксис конфигураций прошли. Развертывание именно этой копии на Ubuntu 24.04, повторный запуск, внешний HTTP, метрики и доставка логов ожидают проверки на отдельной VM.

Архитектура



Состав и версии

Компонент	Реализация
Kubernetes	kubeadm 1.35.9; один control-plane узел с рабочими Pod
Runtime и сеть	containerd.io 2.2.6; Calico 3.32.2, VXLAN, IPv4
Приложение	nginx-unprivileged 1.30.5-alpine; две реплики; Hello World!
Gateway API	Envoy Gateway 1.9.2; CRD 1.6.1; Envoy Proxy 1.39.1
Мониторинг	Prometheus 3.13.3; kube-state-metrics 2.19.1; Nginx exporter 1.5.3
Логирование	Fluentd 1.19.3; постоянный файловый буфер и JSONL архив
Автоматизация	Ansible Core 2.19.3, Makefile, Python; manifests из репозитория

Prometheus устанавливается собственными YAML-манифестами через Ansible. Рекомендуемая VM: 4 vCPU, 8 ГБ RAM, 40 ГБ диска, стабильный IPv4 и исходящий интернет. Метрики и логи хранятся на этой VM.

Исходники и пошаговая инструкция находятся в сопровождающем Git-репозитории.

Обязательные функции

Таблица описывает реализацию и способ приемки. Команды проверки нужно выполнить на чистой Ubuntu Server 24.04 до сдачи решения.

Пункт кейса	Реализация и обоснование	Проверка экспертом
Kubernetes	kubeadm создает кластер на Ubuntu 24.04. Выбор соответствует приоритету кейса и не привязан к облачному провайдеру.	**make bootstrap** **make verify** API, Node, DNS и Pod/Service networking.
Веб-приложение	Две реплики Nginx, Service, probes и лимиты. Простой ответ позволяет однозначно проверить HTTP; приложение пишет access/error-логи.	**make verify-app** HTTP 200, точное тело Hello World!, 404, health и логи.
Gateway API	GatewayClass, Gateway, HTTPRoute и EnvoyProxy. Маршрут по Host demo.local и префиксу / направляет запрос в Service Nginx.	**make verify-gateway** Проверка статусов и NodePort. Выведенный curl повторить с другого компьютера.
Prometheus	Семь jobs и восемь целей: API, Kubelet, CPU/RAM, состояние объектов, Nginx и Envoy. Собственный Prometheus без зависимости от внешнего сервиса.	**make verify-monitoring** Targets, свежие samples, запросы PromQL и вычисление правил.
Fluentd	DaemonSet читает CRI stdout/stderr Nginx, сохраняет исходное сообщение и разобранный access-JSON. Назначение - локальный JSONL архив.	**make verify-logging** Новые запросы через Gateway, совпадение URI, request ID, pod UID и error-записи в архиве.
Ubuntu 24.04	Целевая ОС проверяется preflight; настраиваются systemd, swap, cgroups, runtime и сеть. Проверка на VM еще не проведена.	**make bootstrap** / **make verify-all** Полный запуск, повторный deploy, перезагрузка и внешний HTTP.
Автоматизация	Ansible и Makefile сводят работу к setup, doctor, bootstrap, deploy. Lock-файлы защищают от неявной миграции; diff предшествует apply.	Повторить bootstrap/deploy; проверить отсутствие лишних rollout и сохранение данных. Затем перезагрузить VM.
Документация	README содержит схему, версии, требования, запуск и проверки. Отдельные инструкции описывают приложение, Gateway, метрики, логи и ограничения.	README.md docs/acceptance.md docs/local-validation.json

Для запуска внутри VM добавляйте **INVENTORY=inventory.local.ini** и при необходимости **EXTRA_ARGS='--ask-become-pass'**. С управляющей машины используется настроенный inventory.ini.

Дополнительные возможности

Улучшения представлены конфигурациями и проверками в репозитории. Их эксплуатационные свойства нужно подтвердить на работающем стенде.

Возможность	Как реализована и зачем	Как проверить
Расширенные метрики	CPU/RAM, Ready, реплики, перезапуски, запросы и соединения Nginx, метрики Envoy. Семь правил обнаруживают отказы и исчезновение целей.	verify-monitoring; запросы из docs/monitoring.md; контролируемый отказ.
Проверяемая доставка логов	Уникальные HTTP 200/404 через Gateway сопоставляются с записями в назначении Fluentd. Это отделяет наличие stdout от факта сбора.	verify-logging; отчет содержит access 200/404 и связанный stderr error.
Сохранение данных	Prometheus использует локальный PV с Retain; Fluentd сохраняет позиции и буфер вне Pod. Для лаборатории не нужен внешний storage provisioner.	Замена Pod и перезагрузка VM; запрос к истории метрик и проверка архива до/после.
Ограничение последствий отказа	Nginx работает без root, с read-only rootfs, probes и ресурсными лимитами. У мониторинга отдельные учетные записи и ограниченные роли.	Проверить manifests и runtime securityContext; удалить один Pod Nginx и дождаться восстановления.
Контроль изменений	Версии и ключевые image digest закреплены; upstream-файлы проверяются по SHA-256. Checksums конфигураций управляют rollout.	make lint test; повторный deploy; изменение конфигурации и наблюдение за rollout.

Что подтверждено

Ubuntu 24.04: испытание этой копии еще не проведено. Перед сдачей нужны полный запуск, повторный bootstrap/deploy, перезагрузка и внешний HTTP 200 через Gateway.

Локально: 90 тестов, синтаксис **11 Ansible playbooks**, YAML и контрольные суммы vendor-файлов прошли. Promtool 3.13.3 проверил конфигурацию, семь правил и три сценария. Реальные targets и архив Fluentd нужно проверить на VM.

Короткий путь воспроизведения

make setup
make doctor
make bootstrap
make deploy
make verify-all

Параметры inventory и sudo указаны в README. Общий deploy устанавливает приложение, Gateway, мониторинг и Fluentd, затем выполняет проверки каждого этапа. CI/CD, TLS, Grafana и доставка уведомлений Alertmanager не заявлены.

Ревью и развитие решения

Главная особенность

Проверки связывают инфраструктуру с наблюдаемым результатом: HTTP-ответом, свежими метриками и конкретной записью в архиве логов. Установка, проверка и повторное применение описаны в одном репозитории и не требуют доступа к инфраструктуре участника.

Самое существенное техническое решение

Главный компромисс - способ хранения логов: полный поисковый стек дает удобный поиск, но увеличивает ресурсы и сложность воспроизведения. Выбран Fluentd с локальным JSONL архивом, поскольку кейс допускает такую точку назначения; скрипт приемки проверяет доставку access/error-записей. Цена решения - отсутствие единого поиска и отказоустойчивости при потере VM.

Возможное масштабирование

Несколько узлов и HA. Добавить control-plane и worker-узлы, балансировщик API и резервное копирование etcd; нужны дополнительные VM и отдельные приемочные испытания.

Общее хранение логов. Передавать данные Fluentd в OpenSearch или другое открытое хранилище; потребуются диски, RAM, контроль доступа и политика резервного копирования.

Телеком-сценарии. Разделить сервисы по hostname, добавить метрики задержек и ошибок по маршрутам, SLO и синтетические проверки из разных сегментов сети. Нужны генератор нагрузки и согласованные пороги.

Защита и эксплуатация. Добавить TLS, NetworkPolicy, доставку уведомлений и CI для проверок конфигураций. Потребуются домен/сертификаты, выбранный канал уведомлений и секреты вне Git.

Ограничения текущего профиля

Один узел, IPv4, исходящий интернет; нет HA и резервных копий. История Prometheus ограничена 3 днями / 2 GB блоков, архивы Fluentd - 72 часами; эти настройки не являются дисковой квотой. Fluentd читает hostPath с UID 0 в отдельном namespace; длинные CRI P/F-фрагменты не склеиваются, при сбоях возможны дубли и потери.

Готовность

Конфигурации обязательной части подготовлены, но работоспособность этой копии на Ubuntu 24.04 еще не подтверждена. До сдачи выполните docs/acceptance.md и сохраните результаты. HA, TLS, Grafana, Alertmanager и CI/CD в обязательную часть не входят.

Полный перечень испытаний: docs/acceptance.md. Результаты проверок: docs/local-validation.json. Формат сдачи: docs/submission.md.